

task_bridge

Ваша доска задач и единый источник истины — идеально синхронизированы в обе стороны.

Краткое описание

task_bridge — это универсальный, развязанный, двунаправленный механизм синхронизации задач и досок в Go. Он поддерживает синхронизацию рабочих элементов проекта SQLite (единого источника истины) с трекер-документацией и удалённой доской (первая цель — ClickUp; планируются Jira и Linear) по принципу «последнее изменение побеждает», с предварительным тестированием в сухом режиме и гарантией отсутствия повреждения данных.

Краткая характеристика

Универсальный подмодуль Go, обеспечивающий двунаправленную синхронизацию рабочих элементов SQLite (единого источника истины) ↔ трекер-документации ↔ удалённой доски (сначала ClickUp). Детерминированный принцип «последнее изменение побеждает», предварительное тестирование в сухом режиме, вебхуки с проверкой HMAC; все учётные данные и идентификаторы передаются потребителем во время исполнения.

Подробное описание

Рано или поздно каждая команда ведёт две версии одной и той же работы: реальную — код, документацию, внутреннюю базу данных — и ту, за которой следят менеджеры, например доску в ClickUp. Эти версии расходятся в тот же момент, когда вносится изменение в одну из них, а вручную приводить их в соответствие — утомительная и чреватая ошибками задача, которую никто не выполняет надёжно. task_bridge создан, чтобы устранить этот разрыв, рассматривая все три представления как единую систему, поддерживаемую в синхронном состоянии: **рабочие элементы проекта SQLite**

(единый источник истины), трекер-документацию и удалённую доску (первая поддерживаемая доска — ClickUp; в планах Jira и Linear). Синхронизация детерминирована (последнее изменение побеждает), сначала выполняется в сухом режиме и построена на одном непреложном принципе: она никогда не повредит и не потеряет данные, а также никогда не оставит одну из сторон устаревшей без уведомления. В области, где неосторожная синхронизация может перезаписать неделю работы, такая надёжность — это и есть главная цель. Архитектурно это строгий подмодуль, используемый другими проектами, и он полностью универсален в соответствии с контрактом на развязку, закреплённым в конституции (§11.4.28): он не содержит специфичных для проекта значений, а все учётные данные, идентификаторы досок/папок, поля ключей элементов и пути к базам данных передаются потребителем во время исполнения через `pkg/config.Config`. Модуль имеет чёткую многослойную структуру: CLI (`reconcile/push/pull/resolve/status/conflicts/init`) и долго работающий демон (получатель вебхуков + периодическая синхронизация); тонкая клиентская обёртка над MIT-лицензированным `raksul/go-clickup`; резолвер, преобразующий URL досок/папок в идентификаторы с помощью живых проб API (без угадывания грамматики URL); маппер между локальными рабочими элементами и полями удалённых задач; механизм синхронизации по принципу «последнее изменение побеждает» с явным разрешением конфликтов; получатель вебхуков с проверкой HMAC-SHA256 подписи X-Signature. Модуль честен в оценке своей зрелости: это базовый каркас первой приоритетной версии — структура, интерфейсы, точки входа и граница развязки готовы, но логика синхронизации и вызовы к ClickUp ещё не реализованы (каждый заглушечный метод возвращает явную ошибку «не реализовано» в соответствии с правилом отсутствия фальшивых данных).

Зачем мы его создали

Команды хранят «реальное» состояние работы в коде и документации, а менеджеры живут на досках вроде ClickUp — и эти версии постоянно расходятся. `task_bridge` объединяет их в одну систему, синхронизируя детерминированно и безопасно, чтобы ни одна из сторон не устаревала и не содержала ошибок.

Почему это меняет правила игры

Двусторонняя синхронизация досок обычно реализуется как разовая, жёстко зашитая интеграция, которую каждая команда переписывает кое-как. `task_bridge` переосмысливает

этот подход, превращая его в переиспользуемую библиотеку с инъекцией учётных данных и встроенными гарантиями безопасности данных: предварительный прогон, детерминированный принцип «последнее изменение побеждает», верификация событий через HMAC. Теперь любой проект может внедрить надёжную синхронизацию досок, просто подключив конфигурацию, вместо того чтобы писать очередной хрупкий коннектор, привязанный к внутренней архитектуре.

Что нового

- Трёхсторонняя двунаправленная синхронизация: SQLite (единый источник истины) ↔ трекеры документов ↔ удалённая доска.
- Полная декуплинг (§11.4.28): никаких жёстко зашитых значений проекта — всё подключается в рантайме.
- Динамическое разрешение URL→ID через Live-API вместо ненадёжного парсинга URL-грамматики.
- Верификация входящих вебхуков через HMAC-SHA256 для обработки событий в реальном времени.

Проблемы и решения

- **Безопасность данных при работе с тремя источниками:** решено за счёт детерминированного принципа «последнее изменение побеждает», предварительного прогона и явного разрешения конфликтов.
- **Переиспользуемость без привязки к проекту:** решено через границу инъекции pkg/config (никаких специфичных для проекта данных в коде).
- **Надёжная идентификация досок:** решено путём преобразования URL в ID через динамические запросы Live-API.
- **Честность архитектуры:** решено за счёт того, что заглушки возвращают явные ошибки «не реализовано» (никаких фейковых ответов).

Технологический стек (зачем и как)

- **Go** — движок, CLI (cmd/task_bridge) и демон (cmd/task_bridged).
- **SQLite** — единый источник истины для рабочих элементов.
- **raksul/go-clickup (MIT)** — обёртка для транспорта ClickUp.
- **HMAC-SHA256** — верификация подписей вебхуков.

- **cron + вебхуки** — демон для сверки данных и обработки событий в реальном времени.
- **pkg/config** — граница инъекции учётных данных и ID в рантайме.

Честность статуса: это **P1-каркас** — логика синхронизации ещё не реализована. Не представляйте как готовый продукт.